



EECS 491 Lecture 07

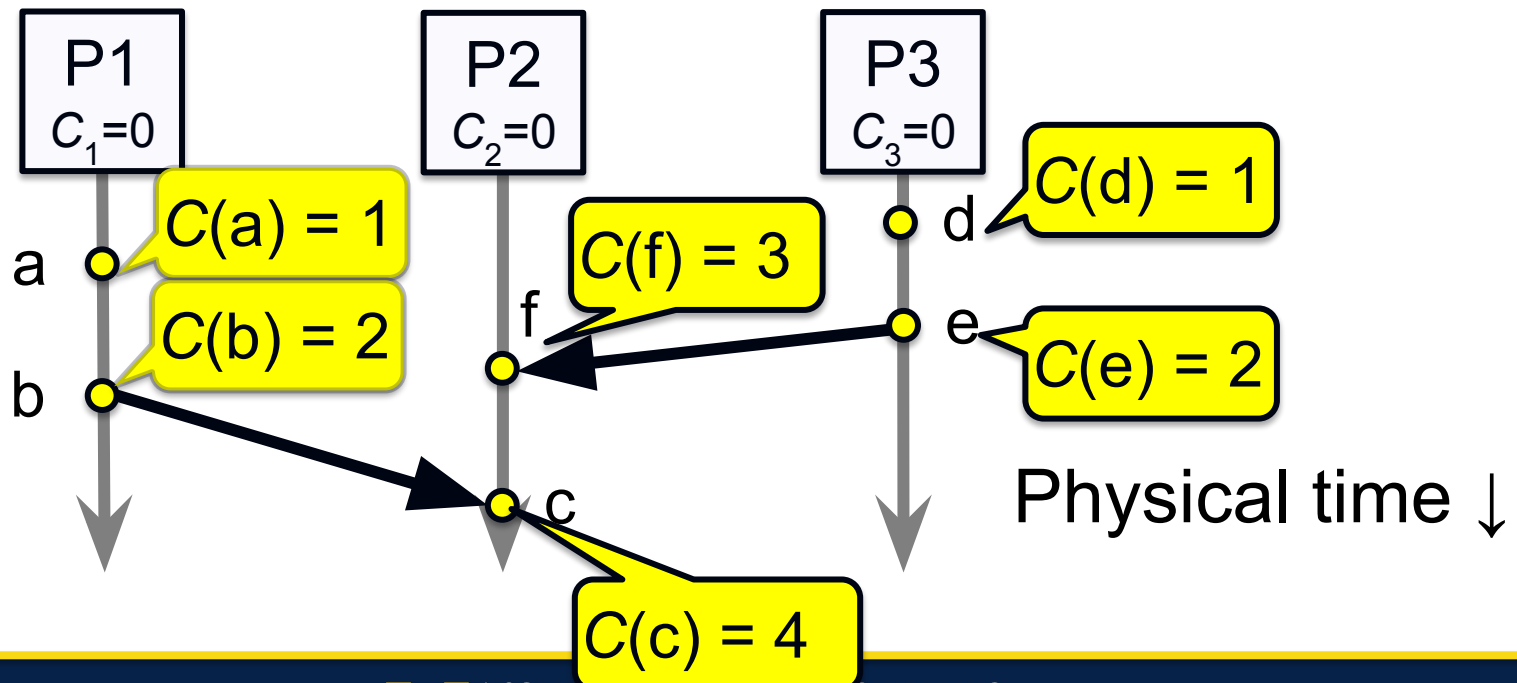
Replicated State Machines, Primary/Backup

Last Time

- Logical clocks
 - guaranteed to preserve causality
 - captures the “the outside world” expectations
 - allowed to arbitrarily order everything else
- Happens-Before ($\rightarrow$) rules
 - Events earlier in process P $\rightarrow$ later ones
 - Sends from P $\rightarrow$ corresponding receives in Q
- Using this to create timestamps
 - Send “current” timestamp with each message
 - Event timestamp must be $1 + \max(\text{predecessors})$

Last Time

1. Before executing an event, $C_j \leq C_i + 1$
2. Send the local clock in any message M
3. On receive, set the clock C_j to
 $1 + \max(C_i, C(m))$
4. Break ties by node ID



Some Consequences

- The name Lamport “Clock” is a misnomer
 - LCs **do not measure the passage of time**
 - they measure the passage of (causal) **events**
- The straightforward implementation of LC
 - counts length of the longest causal path to “T=0”
- Nodes that do not communicate
 - can have “clocks” that drift arbitrarily far
 - but cannot **observably** influence each other
 - so it **doesn’t matter** which order we pick
 - as long as we pick a consistent one

What does “influence” mean?

- Causality tracks what **observers expect**
 - we will formalize this idea (later)
- Informally, if an observer:
 - asks for an operation, and is told that it happened
 - expects that outcome to be reflected in “later” events
 - causality allows us to capture this
- Concurrent events can *interact* non-causally
 - User 1: deposit || User 2: pay interest
 - these are not causally related
 - but they must be applied in the same order everywhere!

Today

- How can we use this idea in RSMs?
- Key idea
 - Every operation O has some timestamp T
 - Apply O when we know
 - all other replicas have advanced **at least to** T
 - implies we already know all updates U such that $U \sqsubseteq O$
- How do we know that?
 - receive a message with a send time $S \geq T$
 - (...if **all** messages arrive, and arrive in order...)

Today

- Applying this to replicated state
 - including a detailed example
- Problem: cannot make progress with failures!
- Handling (message or node) failures
 - Primary/Backup Replication (part 1)

RSM with Lamport Clocks

- How do all replicas execute operations...
 - ...in the same order...
 - ...by using Lamport Clock value?
- Remember that *so far*
 - nodes do not fail
 - messages are delivered reliably
- We will relax these shortly

RSM with Lamport Clocks

- How do all replicas execute operations...
 - ...in the same order...
 - ...by using Lamport Clock value?
- Key idea
 - always send “new” updates to peers with next message
 - queue of pending updates, ordered by clock value
 - apply head element when no earlier updates are possible
- We know earlier updates are impossible when:
 - we know that all other clocks are at least as far as head element
 - so we must know all earlier updates at all other nodes
- *Assuming in-order delivery*

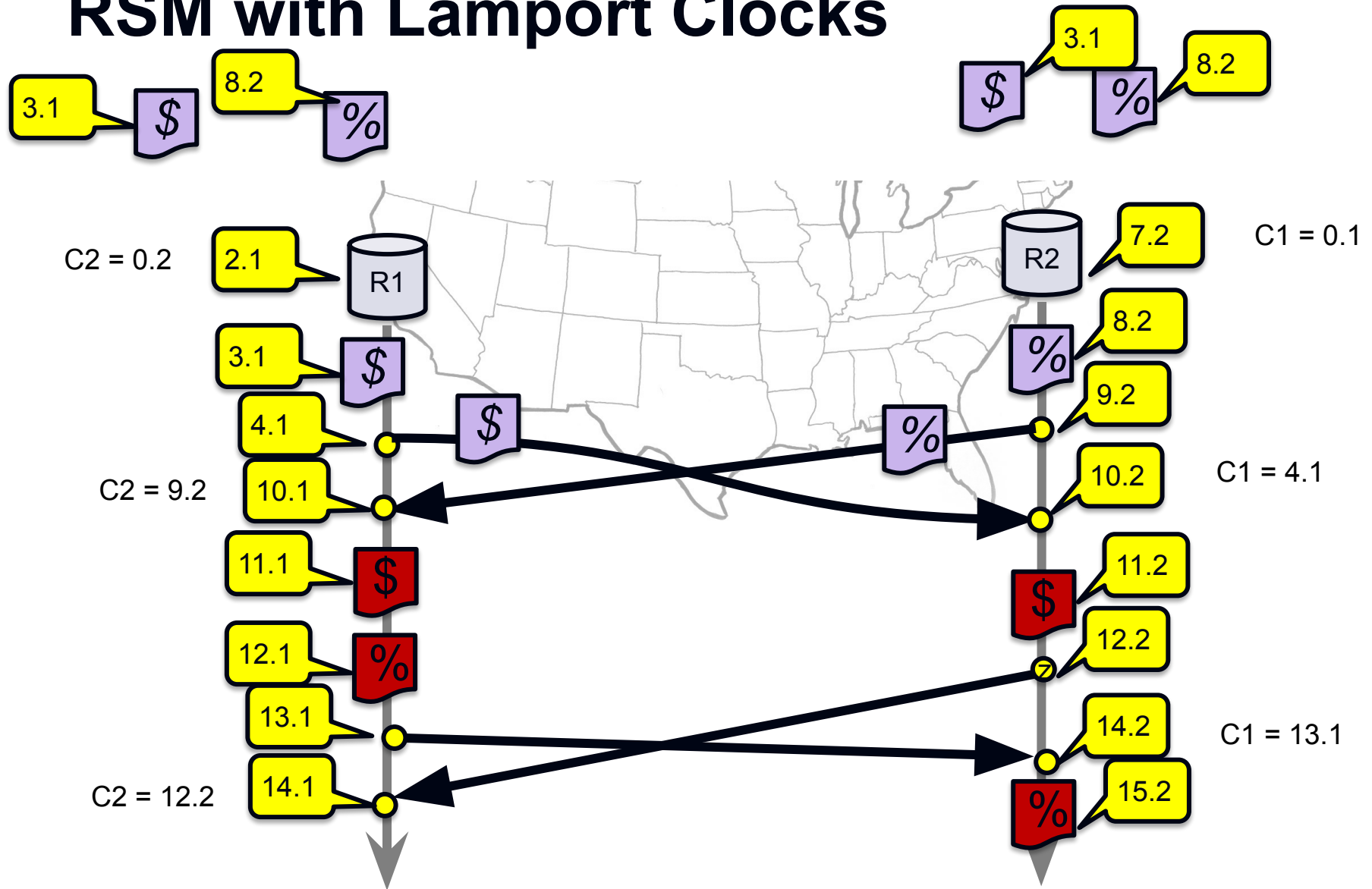
Conventions

- For these diagrams:

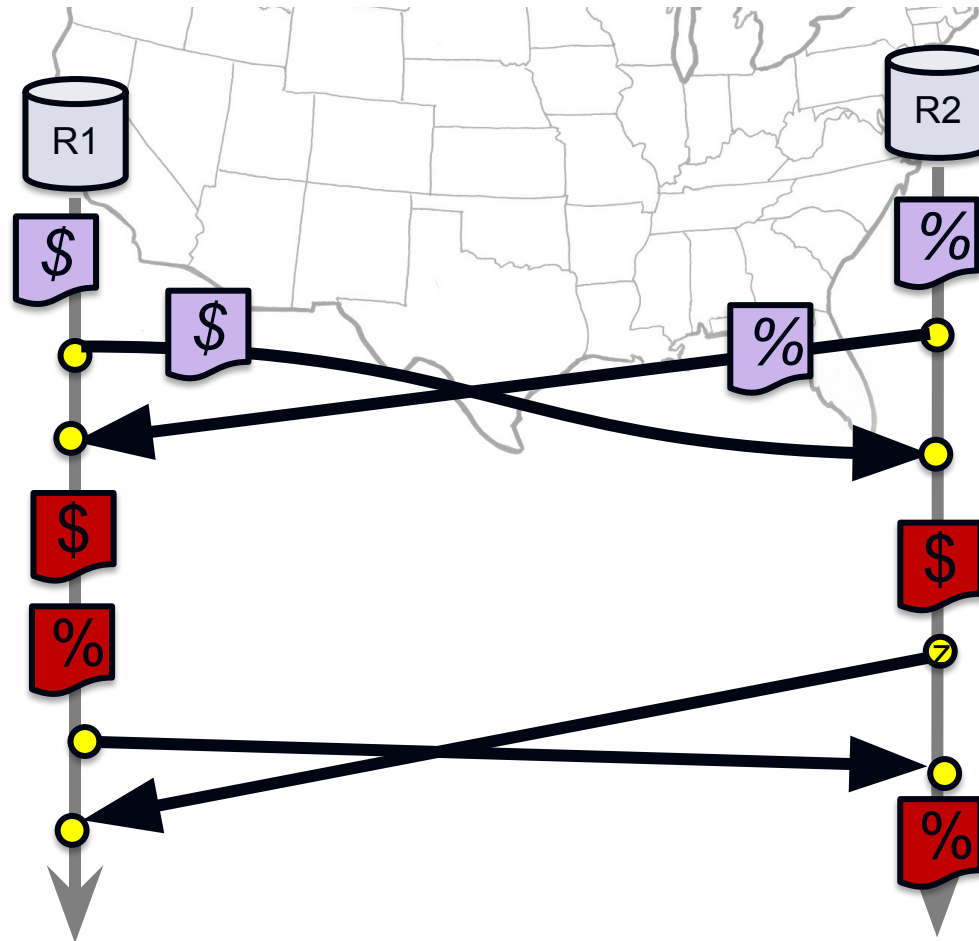
- proposed updates in light purple
- committed updates in red
- every event has its own timestamp



RSM with Lamport Clocks



RSM with Lamport Clocks



Reducing Waiting

- May have to wait arbitrarily long
 - if you do not get any messages from other replicas
- Heartbeat messages
 - periodically exchange messages
 - no data, only clock updates (+ proof-of-life)
- Can sometimes tolerate temporary inconsistency
 - apply updates immediately
 - maintain undo log to roll back **if necessary**

Jeff Dean's (Dated) “Numbers to Know”

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns

“If necessary”

- Lamport clock: $a \rightarrow b$ implies $C(a) < C(b)$
 - converse not necessarily true
 - $C(a) < C(b)$ means either $a \rightarrow b$ or $a \parallel b$
- Physical clocks have this problem too!
- May be sufficient to order causal updates
 - many social media apps can use this
 - feed can change inconsistently over time; little harm
 - e.g. it's okay if concurrent comments seen in different orders
- Not always!
 - e.g. bank balance updates **must** be in **one order**

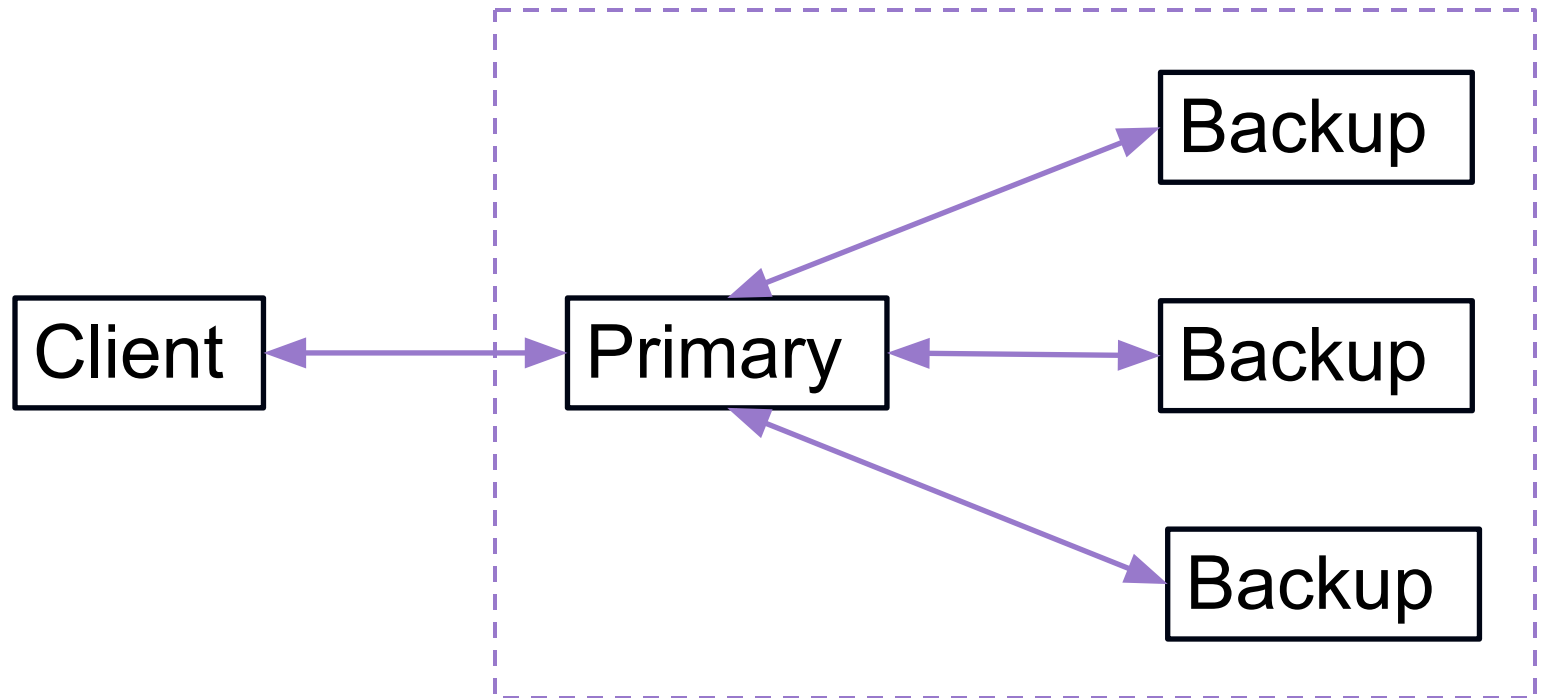
What About Failures?

- Replicas can't execute update at time T until
 - clocks at all other replicas known to be at least T
- What does that imply for failures?
 - while one replica is down, all others are blocked!
- Failures last an arbitrary period of time
 - Lamport clocks a dead end (but only for now...)
- Try something new

A few burning questions

- Internally: channels are a pointer to the queue
 - copying a channel DOES NOT copy the queue
- What if we improved the network?
 - does that make clock synchronization easier?
 - it would, but there is no incentive to do so!
 - ...unless one company owned it all
- If I don't have a PhD, how do I keep up?
 - the same way you do if you do have one!

Primary/Backup Replication



Primary/Backup Replication

- How to handle primary failure?
 - Promote one of the backups as the primary
- How to handle backup failure?
 - Add another machine as a backup

Primary/Backup Synchronization

- When should primary sync up with backups?
- What should be transferred when syncing?

Example: MapReduce Manager

```
RegisterServer() {  
    Primary & Backups in sync  
    while (1) {  
        receive msg and parse addr  
        pick task to assign  
        mark task as assigned  
        respond with task assignment  
    }  
}
```

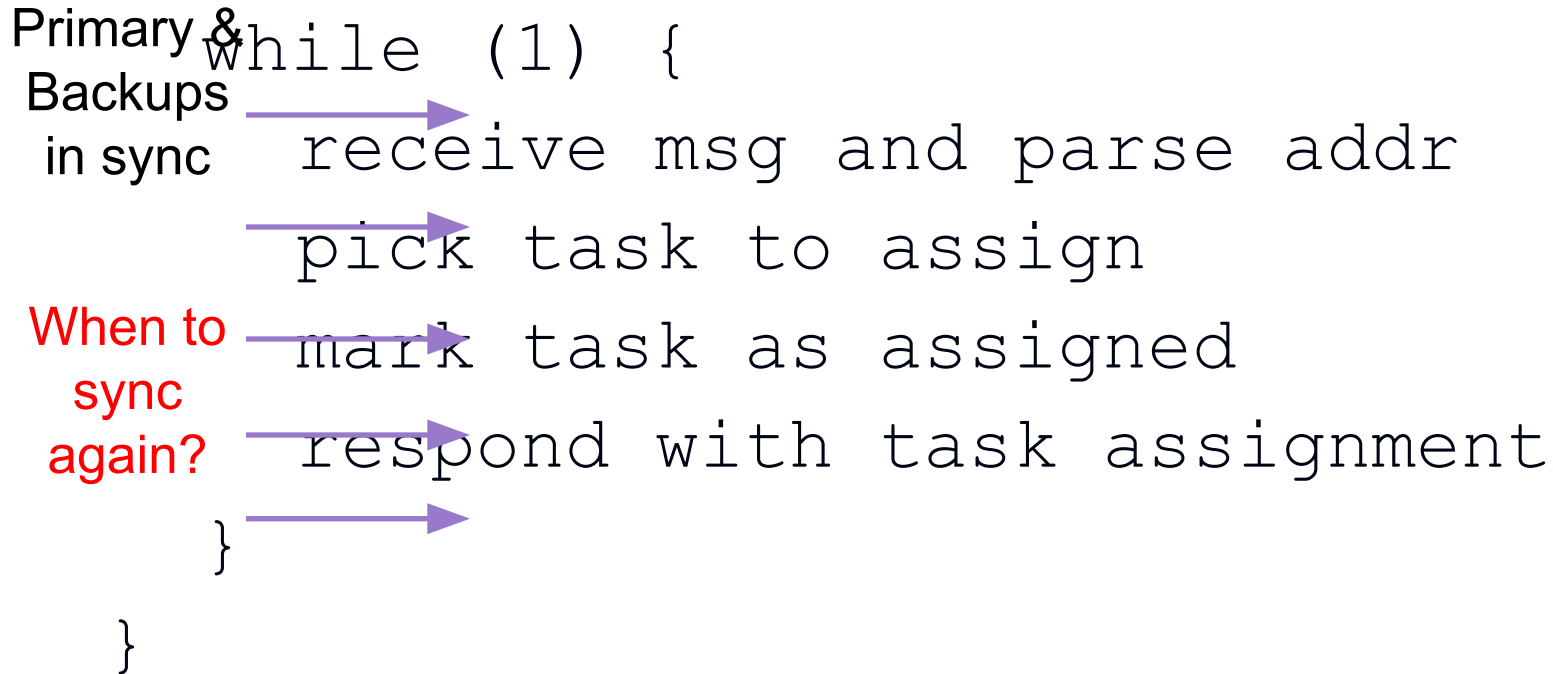
Primary & Backups in sync

When to sync again?

Example: MapReduce Manager

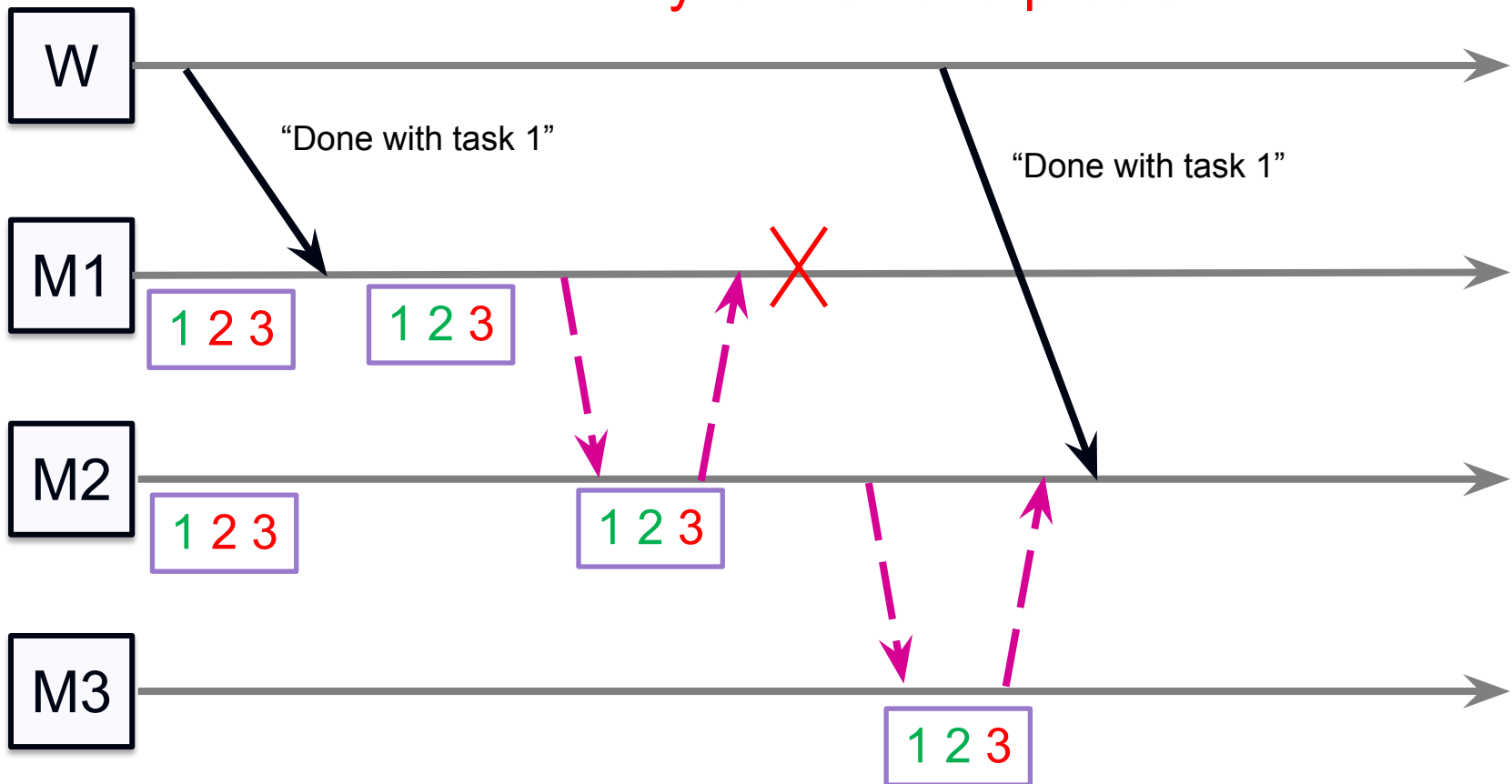
```
RegisterServer() {  
    Primary & Backups in sync  
    while (1) {  
        receive msg and parse addr  
        pick task to assign  
        mark task as assigned  
        respond with task assignment  
    }  
}
```

When to sync again?

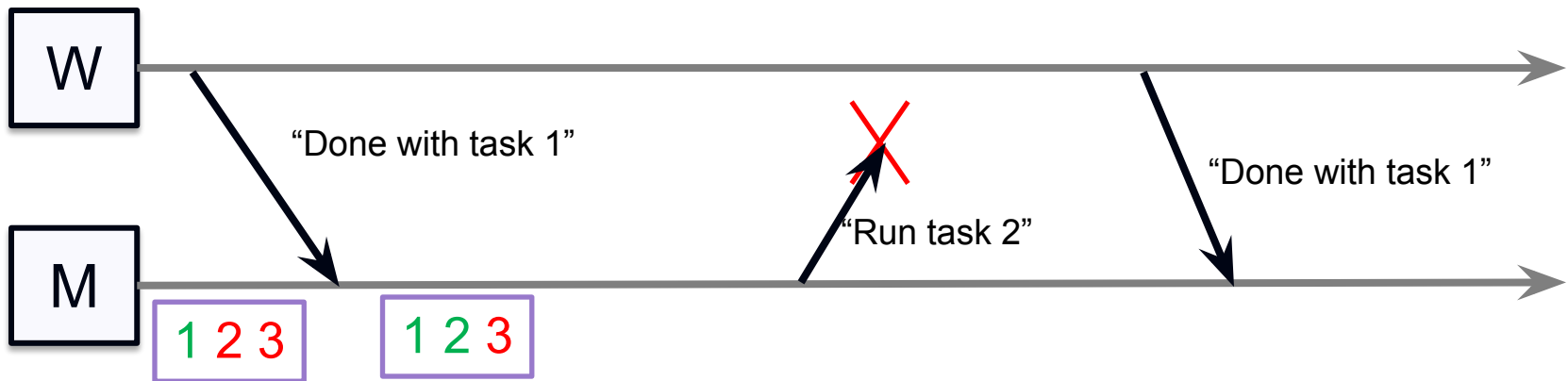


Primary/Backup Sync

Why is this not a problem?



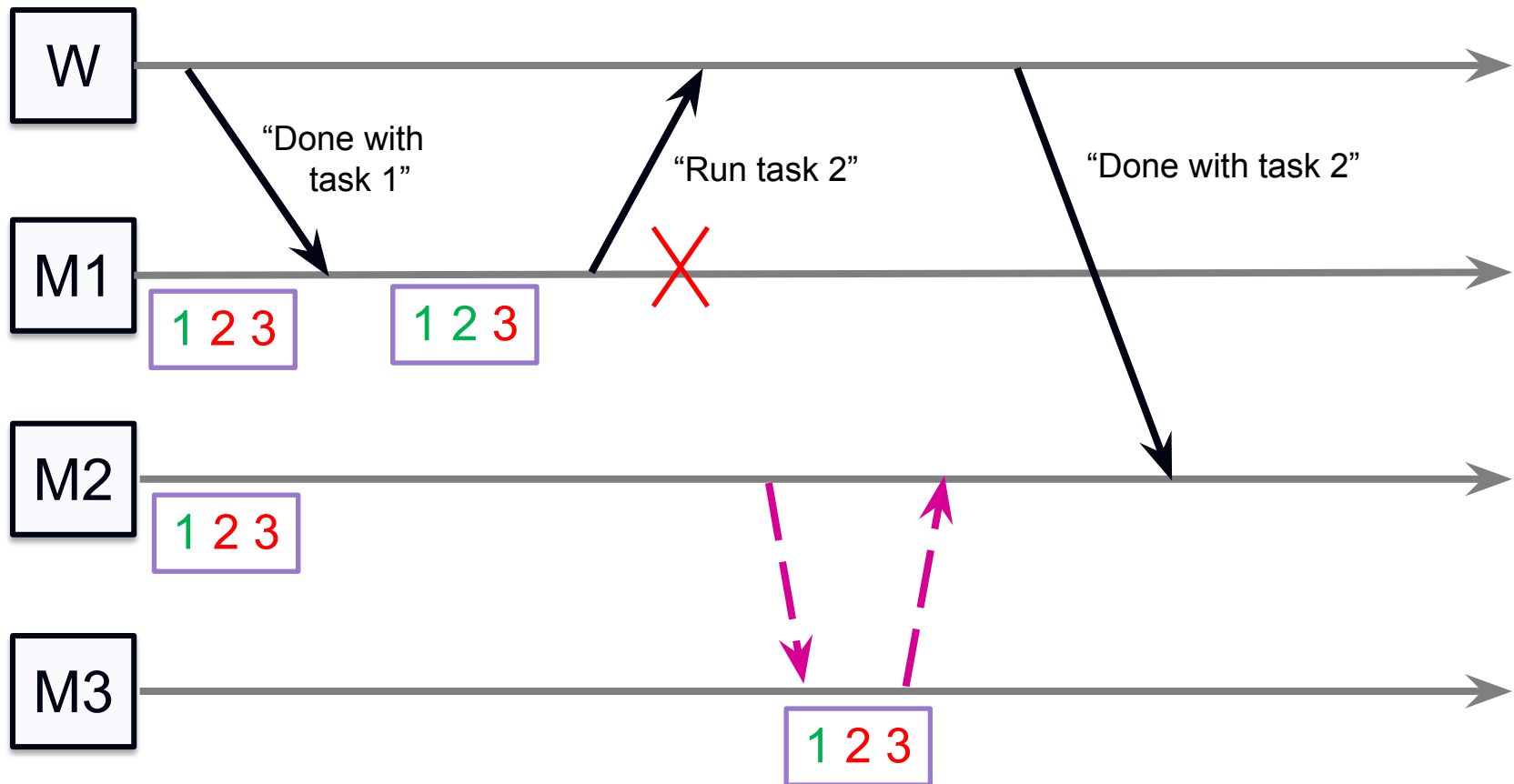
Same “Problem” Without Replication



Example: MapReduce Manager

```
RegisterServer() {  
    Primary & Backups in sync → while (1) {  
        receive msg and parse addr  
        pick task to assign  
        mark task as assigned  
        When to sync again? → respond with task assignment  
    } →  
}
```

Primary/Backup Sync



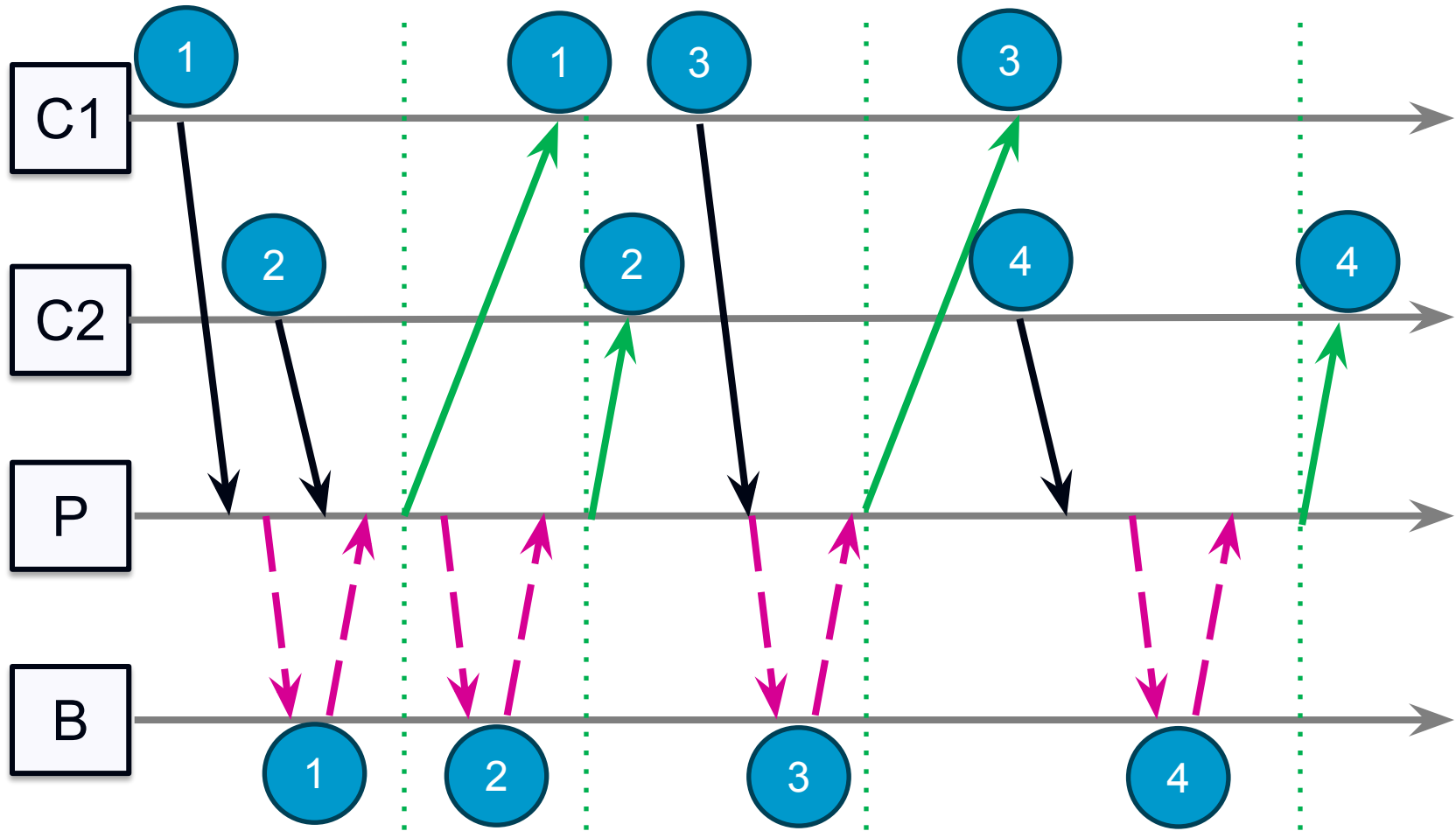
Primary/Backup Sync



External Consistency

- The thing that matters is **visibility**
- An update that could be visible to the client
 - must be stored on the primary *and* the backup
- Primary/Backup inconsistency is allowed
 - but only until the update is potentially visible
- Informally
 - everything a client might know about the primary
 - must also be known by the backup

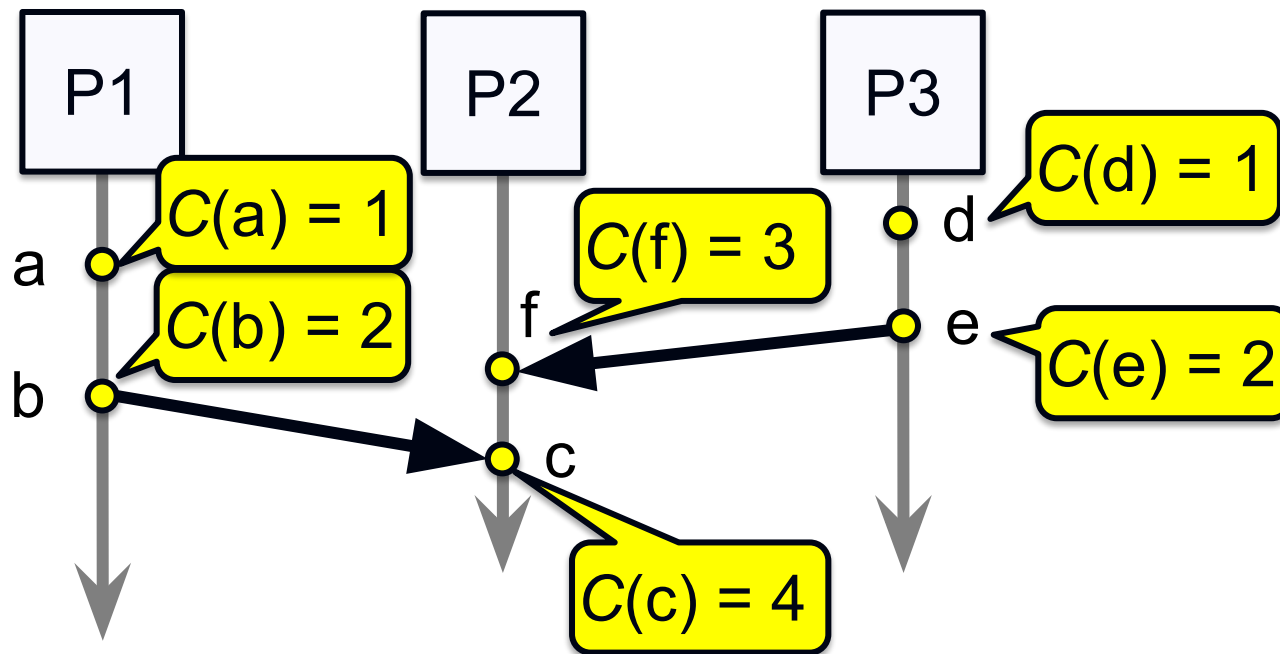
Primary/Backup Sync



Next Time

- More on Primary/Backup

Lamport Clocks



Vector Clocks

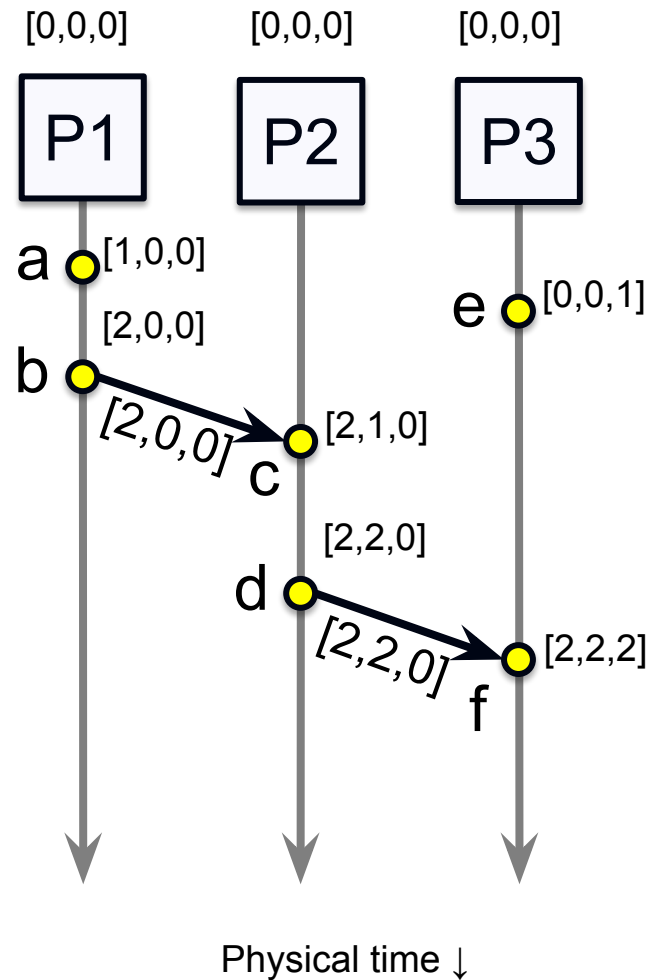
- Label each event **e** with a vector
 - $V(\mathbf{e}) = [c_1, c_2, \dots, c_n]$
 - No. of components = No. of processes
- Semantic interpretation:
 - c_i is a count of events in process i that **causally precede e**

Updating vector clocks

- Initially, all vectors are $[0, 0, \dots, 0]$
- Two rules for updating
 1. For each **local event** on process i
 - increment local entry c_i
 2. If process j **receives** message with vector $[d_1, d_2, \dots, d_n]$:
 - 1: set each local entry $c_k = \max\{c_k, d_k\}$
 - 2: then increment local entry c_j

Vector Clock: Example

- All counters start at $[0, 0, 0]$
- Applying local update rule
- Applying message rule
 - For each component, compute max of local clock and that in message
 - Increment component for local process



Vector Clocks Establish Causality

- Event a happens-before event b if and only if
 - $V(a) \neq V(b)$ *and*
 - $a_k \leq b_k$ for all k
 - written as $V(a) < V(b)$
- Two events are concurrent if
 - there exist two different indexes i, j such that
 - $a_i < b_i$ and $a_j > b_j$
- $V(a) < V(b)$ if and only if
 - they are linked by a chain of events

Vector Clock: Example

